

Main Scripts:

- **main_cell_segmentation.py**
- **main_label_images.py**
- **main_label_images_to_omero.py**
- **main_biomarker_tables.py**
- **main_biomarker_tables_to_omero.py**
- **main_cell_segmentation_analysis.py**
- **Main_celltype_cluster_tables.py**

main_cell_segmentation.py

- Environment:

```
module load condaenvs/new/deepcell
```

- This can be run with **main_cell_segmentation.sh**
- What does it do?
 - **get_top5_means_ecadherin**
 - gets the means of the top 5% of pixels in every tile, only in the ecadherin channel and saves it as a npy, also saves tile positions and tile sample names
 - **cluster_tiles**
 - performs kmeans clustering on the array of ecadherin means for each sample.
 - uses n_clusters_preprocessing (= 3)
 - these clusters rep ecad+ and ecad- tissue which undergo diff forms of cell segmentation
 - **create_cluster_masks**
 - creates both a black and white mask per cluster and a colored mask with both clusters for each sample.
 - **new_positions**
 - the 2 clusters with the higher cluster centers (cluster mean) are assigned as ecad+, the cluster with the lowest cluster center (cluster mean) are assigned as ecad-
 - assigns the tile positions as either ecad+ or ecad-. saves these tile positions as separate csvs to be called during cell segmentation
 - **get_cell_segmentations**
 - generates cell segmentations and creates a cell by marker matrix + corresponding metadata
 - ecad+: whole cell segmentation
 - ecad-: nuclear segmentation w/pixel expansion = 2
 - metadata columns
 - cell_label, centroid_x, centroid_y, area, perimeter, axis_ratio, tile_x, tile_y, slide_id, tissue_type
 - what is saved by this function:
 - matrix
 - metadata

- cell_sample_names
- tile_sample_names
- tile_positions
- segmentation_masks
- separate versions of all the data saved are created for ecad+ and ecad-
- matrix, metadata and cell_sample_names are concatenated and reordered into 1 final version

main_label_images.py:

- Environment:

```
conda activate omero #have to be in proteome group to access
```

- this can be run with **main_label_images.sh**
- What does it do?
 - **split_data_by_sample**
 - splits the segmentation_masks, tile_positions and raw_metadata by sample
 - **only** the following is saved in metadata: cell_label, tile_x, tile_y, slide_id, label_image_cell_index
 - **create_label_image**
 - **create_label_tiles** - updates the segmentation masks so each cell is represented by the label_image_index not just the cell_label for that tile
 - **create_label_slide** - retiles the image back to its original size and shape and saves it as a tiff
 - **convert_tiff_to_zarr** - converts label_image.tiff to label_image.zarr with bioformats2raw

main_label_images_to_omero.py

- *this script has to be run in datamover node with research drive already mounted via terminal so the script can access the research drive*
- Environment:

```
source bash_scripts/.env #uses kerberos and password
conda activate omero
```

- this can be run with **main_label_images_to_omero.sh**
 - first mount the drive!! then run script
- What does it do?
 - moves the label_image.zarr to the mounted research drive
 - creates a backup.zarr in the research drive
 - runs roi_converter_ngff to get the label_image.zarr uploaded to omero
- After this is run: the ROIs of each image need to be added to **omero_image_dict** in the **config** file before continuing

main_biomarker_tables.py

- Environment:

```
module load condaenvs/new/deepcell
```

- this can be run with **main_biomarker_cluster_tables.sh**
- needs to be run after `main_label_images_to_omero.py` because need the ROIs to create the omero table
- What does it do?
 - **lineage_marker_cluster_tables**
 - clusters the cell means for the lineage markers we will be thresholding, and saves them in a dataframe per sample that is compatible with omero
 - cluster_tables are saved for each sample in the following format:
 - object (1 $\rightarrow$ len(df) + 1)
 - roi - from omero_image_dict
 - channel name with clusters identity per cell (0 $\rightarrow$ n_clusters-1)
 - This is only done for lineage markers that we think need to be thresholded to improve the accuracy of our results.
 - these channels are defined in the **config** under:
threshold_channel_names
 - **biomarker_mean_tables**
 - separates the biomarker cell means (from matrix) per sample and and saves them in a dataframe per sample that is compatible with omero
 - cluster_tables are saved for each sample in the following format:
 - object (1 $\rightarrow$ len(df) + 1)
 - roi - from omero_image_dict
 - channel name with clusters identity per cell (0 $\rightarrow$ n_clusters-1)
 - This is done for all channels as its just good to have for later analysis

main_biomarker_tables_to_omero.py

- Environment:

```
source bash_scripts/.env #uses kerberos and password
conda activate omero
```

- this can be run with **main_biomarker_tables_to_omero.sh**
- uploads the biomarker tables created in **main_biomarker_tables.py** to omero
 - for each sample the following is uploaded:
 - **lineage_markers_{n_clusters}clusters.csv**
 - **raw_biomarker_means.csv**
- After this is run: the clusters (0 $\rightarrow$ n_clusters-1) need to be designated as positive or negative in the **lineage_markers_cluster_dict** in the **config**

main_cell_segmentation_analysis.py

- Environment:

```
module load condaenvs/new/deepcell
```

- this can be run with **main_cell_segmentation.sh**
- what does it do?

- **threshold_markers**
 - thresholds certain cell lineage markers - this was done with kmeans
- **get_normalized_matrix** - filters and normalizes matrix
 - optional: filters by sample - (output stored in out_*/matrix_filtered_markers)
 - this is only created if samples_to_remove is not None
 - filters by dapi distribution (cells below a threshold of DAPI)
 - also saves filtered metadata and cell sample names
 - normalizes matrix per sample per biomarker
 - cap each biomarker to 99.9% value
 - zscore + arcsinh per sample!
 - filters by biomarker to only include cell lineage markers (*could filter by biomarker first when filtering by sample*)
- **gen_umap_embedding.plot_umap**
 - generates umap of normalized cell lineage marker matrix
- **kmeans.clustering**
 - performs kmeans - cluster labels are saved 1 - n_clusters (**NOT 0 - n_clusters -1**)
- **kmeans.add_labels_to_metadata**
 - **only** saves these metadata cols: index, slide_id and cluster label
 - uses the raw unfiltered metadata as a base and assigns 0s wherever there is no cluster label (since that row was filtered out)
- **clustering_analysis** - generates analysis figs of the results
 - color umap by sample (see if batch effect)
 - color umap by marker (ALL MARKERS)
 - color umap by cluster
 - heatmap of mean expression of lineage biomarkers per cluster
 - heatmap of mean expression of all biomarkers per cluster
 - heatmap of the proportion of each sample in each cluster (see if batch effect)
- If something was already generated, the script will skip those steps until it hits something that has not been generated yet
- if you run the pipeline with x number of clusters, and then rerun with y number of clusters, it will skip segmentation, normalization, and umap creation and just create new a new folder for y_clusters under both the clustering and analysis_figures directories

main_cell_segmentation_analysis.py

- needs to be run after main_cell_segmentation_analysis.py because of the environment
- Environment:

```
source bash_scripts/.env #uses kerberos and password
conda activate omero
```

- this can be run with **main_celltype_cluster_tables.sh**
- What does it do?

- Creates and uploads an omero table with the cluster information (clusters are representing cell types)
- Omero_tables are saved for each sample in the following format:
 - object (1 → len(df) + 1)
 - roi - from omero_image_dict
 - celltype cluster info
- The same date used when running main_cell_segmentation_analysis.py is used in the omero table name when uploaded to omero to track which version